

Projet d'Intelligence Artificielle

Implémentation d'une IA pour le jeu Reversi (Othello)

Ibrahim Saleh Arsil, Mahdjoub Amélia, Delucinge Thibaud

Licence MIASHS – Parcours Cognition

Université Grenoble Alpes

Semestre 5 – 2025

Table des matières

1	Introduction	3
2	Reversi	3
2.1	Règles du jeu	3
2.2	Propriétés du jeu	4
3	Architecture et implémentation	4
3.1	Organisation générale du projet	4
3.2	Représentation du plateau	5
4	Algorithme de décision	5
4.1	MiniMax et NegaMax	5
4.2	Élagage Alpha-Bêta	6
4.3	Move Ordering	6
5	Fonction d'évaluation heuristique	7
5.1	Découpage en phases de jeu	7
5.2	Heuristiques utilisées	7
5.3	Pondération des heuristiques	7
6	Analyse de performances	8
6.1	Temps de calcul	8
6.2	Taux de victoire IA vs Random	9

6.3	Comparaison entre profondeurs successives	10
6.4	Consommation mémoire	10
7	Tournoi IA vs IA	11
7.1	Méthodologie expérimentale	11
7.2	Résultats	11
8	Conclusion	11
9	Bibliographie	13

1 Introduction

Ce projet s’inscrit dans le cours d’Intelligence Artificielle (Semestre 5) de la Licence MIASHS. L’objectif est de développer une IA capable de jouer au Reversi (Othello), avec une profondeur de recherche configurable.

Par souci de praticabilité, trois modes de jeu ont été implémentés :

- Humain vs Humain
- Humain vs IA
- IA random vs Human
- IA random vs IA
- IA vs IA

Le jeu étant à information parfaite, il est crucial d’anticiper les coups adverses. Le cœur du projet repose sur l’algorithme MiniMax (implémenté ici sous forme de NegaMax) avec élagage Alpha-Bêta, ainsi que sur une fonction d’évaluation basée sur des heuristiques classiques.

Enfin, la stratégie valorise fortement les **coins** et les **bords**, car ces positions sont généralement stables et difficiles à reprendre.

Un tournoi entre de diverses stratégies est ensuite mené pour déterminer la plus efficace, et en déduire les meilleurs valeurs des paramètres de la fonction heuristiques.

Ce rapport présente l’architecture du projet, les choix algorithmiques, les heuristiques, ainsi qu’une analyse expérimentale (temps, mémoire, taux de victoire).

2 Reversi

2.1 Règles du jeu

Reversi est un jeu combinatoire à deux joueurs sur un plateau 8×8 . Au début, quatre pions sont placés au centre : deux blancs (D4, E5) et deux noirs (D5, E4). Le but est d’avoir, à la fin, plus de pions de sa couleur.

Un coup est légal s’il encadre une ligne continue de pions adverses entre le pion posé et un pion déjà présent du joueur courant (horizontalement, verticalement ou en diagonale). Les pions encadrés sont alors retournés.¹

Si un joueur n’a aucun coup légal, il passe son tour. La partie se termine lorsque : aucun des deux joueurs ne peut jouer, ou le plateau est rempli.

1. <https://www.ffothello.org/othello/regles-du-jeu/>

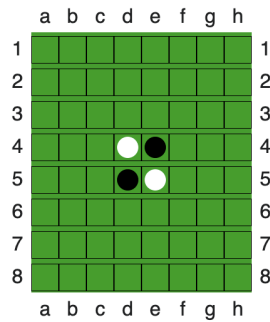


FIGURE 1 – Position initiale.

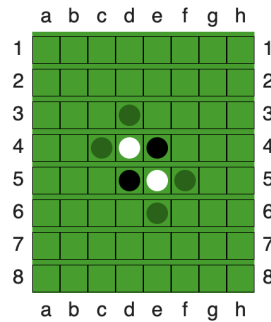


FIGURE 2 – Exemple : coup des blancs.

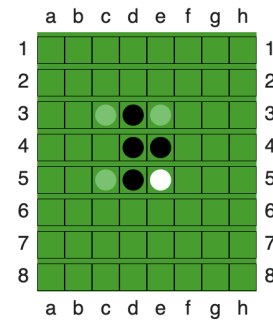


FIGURE 3 – Exemple : coup des noirs.

2.2 Propriétés du jeu

Reversi est un jeu à information complète et parfaite car « tous les joueurs disposent à tout moment des mêmes informations pour effectuer leurs choix » [3]. Il est également déterministe : l'aléatoire n'intervient pas. Enfin, il s'agit également d'un jeu à somme nulle « où la somme des gains et des pertes de tous les participants est égale à zéro. Cela signifie donc que le gain de l'un constitue obligatoirement une perte pour l'autre » [4].

Ces propriétés rendent Reversi particulièrement adapté à l'utilisation d'algorithmes de recherche adversariale tels que MiniMax et NegaMax.

3 Architecture et implémentation

3.1 Organisation générale du projet

```
projet_reversi_2025/
main.py
ai/
  ai_profiles.py
  heuristics.py
  heuristics_consts.py
  heuristics_explained.txt
  minimax.py
benchmarks/
  benchmark.py
game/
  board.py
  game_manager.py
  player.py
ui/
  game_settings.py
```

```

game_sign.py
messages.py
.venv/ (env)

```

Le projet a été structuré sous différents modules afin de séparer les fonctionnalités principales. Le module `game` gère la logique du jeu, le plateau et les règles, le module `ai` contient l'algorithme de décision et les heuristiques. Enfin, le module `ui` est dédié à l'affichage dans la console et à l'interaction avec l'utilisateur.

3.2 Représentation du plateau

Le plateau est représenté par une matrice 8×8 de valeurs $\{-1, 0, +1\}$:

$$\begin{cases} +1 : & \text{pion Bleu} \\ -1 : & \text{pion Rose} \\ 0 : & \text{case vide} \end{cases}$$

Cette représentation simple permet un accès rapide aux informations nécessaires à la génération des coups légaux et à l'évaluation des positions.

Pour l'affichage console, la bibliothèque `rich`² permet d'obtenir un rendu lisible avec des couleurs distinctes pour chaque joueur, tout en restant compatible avec des tests automatisés..

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & +1 & 0 & 0 & 0 \\ 0 & 0 & 0 & +1 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

FIGURE 4 – Représentation interne (matrice).

	A	B	C	D	E	F	G	H
1	.	.	.	.	.	.	.	.
2	.	.	.	.	.	.	.	.
3	.	.	.	.	.	.	.	.
4	.	.	.	●	●	.	.	.
5	.	.	.	●	●	.	.	.
6	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.

FIGURE 5 – Affichage côté joueur (terminal).

4 Algorithme de décision

4.1 MiniMax et NegaMax

L'algorithme MiniMax « s'applique à la théorie des jeux pour les jeux à deux joueurs à somme nulle (et à information complète) consistant à minimiser la perte maximum

2. <https://github.com/Textualize/rich>

(c'est-à-dire dans le pire des cas) » [1].

NegaMax considère que maximiser son propre score revient à minimiser celui de l'adversaire. Cela permet de simplifier l'implémentation, tout en conservant la logique du MiniMax classique.

On s'appuie donc sur l'identité suivante [2] :

$$\min(a, b) = -\max(-a, -b)$$

Ainsi, évaluer une position pour le joueur courant revient à prendre l'opposé de la valeur évaluée pour l'adversaire. La fonction de score utilisée par NegaMax s'écrit alors :

$$\text{score}(s) = \max_{m \in \text{coups}(s)} (-\text{score}(s \rightarrow m))$$

4.2 Élagage Alpha-Bêta

L'exploration exhaustive de l'arbre étant trop coûteuse, un élagage Alpha-Bêta est utilisé. Il conserve les bornes α (meilleur score garanti) et β (pire score acceptable) et coupe les branches inutiles :

$$\alpha \geq \beta \Rightarrow \text{coupure}$$

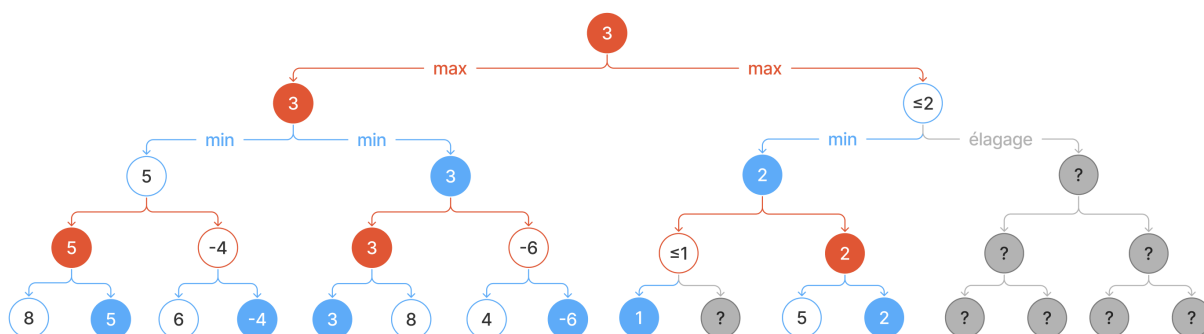


FIGURE 6 – Principe de l'élagage Alpha-Bêta.

4.3 Move Ordering

L'élagage Alpha-Bêta n'active le tri des coups qu'en phase *midgame* et pour des profondeurs ≥ 4 . Avant l'exploration, chaque coup est noté avec une heuristique plus légère/rapide puis trié par score décroissant, afin d'augmenter les coupures.

Heuristique de tri (quick_eval) :

- Gros bonus si le coup prend un coin
- Forte pénalité si la case est dangereuse (adjacente à un coin libre, X/C-square)
- Petit bonus pour les bords
- Bonus proportionnel au nombre de pions retournés ($n_{pions} \times 10$).

5 Fonction d'évaluation heuristique

5.1 Découpage en phases de jeu

Une partie de Reversi est divisée en 3 phases : l'ouverture, le milieu et la fin de la partie. Selon la phase, il y a une évolution des paramètres de jeu à prendre en compte.

Durant l'ouverture, le nombre de choix possibles et la valeur des positions sont les plus importants.

En milieu de partie, il y a intérêt à minimiser les risques et de capturer les coins.

En fin de jeu c'est plutôt la quantité de pions à maximiser qui est essentiel .

5.2 Heuristiques utilisées

L'évaluation estime la qualité d'une position en fonction de plusieurs critères (pondérés selon la phase de jeu) :

- Mobilité (nombre de coups possibles)
- Coins et positions stables
- Risque près des coins (cases X/C)
- Frontière (pions exposés)
- PST (Positionnal Score Table)
- Différence de pions (plutôt utile en fin de partie)

5.3 Pondération des heuristiques

Chaque heuristique est pondérée différemment selon la phase de jeu car les priorités évoluent au fil de la partie. Ces poids ont été déterminés via leur ordre de grandeur à partir de la littérature existante sur Reversi :

$$\left\{ \begin{array}{l} \text{Open : } disc : 0.10 < risk = frontier : 0.75 < pst : 1.00 < mobility : 1.25 < corners : 1.50 \\ \text{Mid : } disc = risk : 0.50 < frontier = pst : 1.00 < mobility : 1.25 < corners : 1.50 \\ \text{Late : } mobility = risk = frontier = pst : 0.50 < corners : 1.25 < discs : 1.50 \end{array} \right.$$

En variant la valeur des poids, 4 nouvelles stratégies sont obtenues pour l'organisation du tournoi IA vs IA :

- Defense : minimisation des risques ,
- Corners : priorité de jouer les coins ,
- Mobility : favorise le nombre de coups possible ,
- End : tout se joue en fin de partie .

Stratégie	Phase	Mobility	Corner	Risk	Frontier	PST	Discs
Default	Opening	1.25	1.50	0.75	0.75	1.00	0.10
	Midgame	1.25	1.50	0.50	1.00	1.00	0.50
	Endgame	0.50	1.25	0.50	0.50	0.50	1.50
Defense	Opening	1.25	1.50	5.00	0.75	1.00	0.10
	Midgame	1.25	1.50	5.00	1.00	1.00	0.50
	Endgame	0.50	1.25	0.50	0.50	0.50	1.50
Corners	Opening	1.25	5.00	2.50	0.75	1.00	0.10
	Midgame	1.25	5.00	2.50	1.00	1.00	0.50
	Endgame	0.50	5.00	2.50	0.50	0.50	1.50
Mobility	Opening	5.00	1.50	0.75	0.75	2.50	0.10
	Midgame	5.00	1.50	0.50	1.00	2.50	0.50
	Endgame	0.50	1.25	0.50	0.50	0.50	1.50
Endgame	Opening	1.25	1.50	0.75	0.75	1.00	0.10
	Midgame	1.25	1.50	0.50	1.00	1.00	0.50
	Endgame	0.50	2.00	0.50	0.50	1.00	5.00

TABLE 1 – Poids des heuristiques utilisés par chaque stratégie selon la phase de jeu (opening, midgame, endgame).

6 Analyse de performances

6.1 Temps de calcul

Le temps moyen de calcul par coup a été mesuré en fonction de la profondeur de recherche et de la phase de jeu.

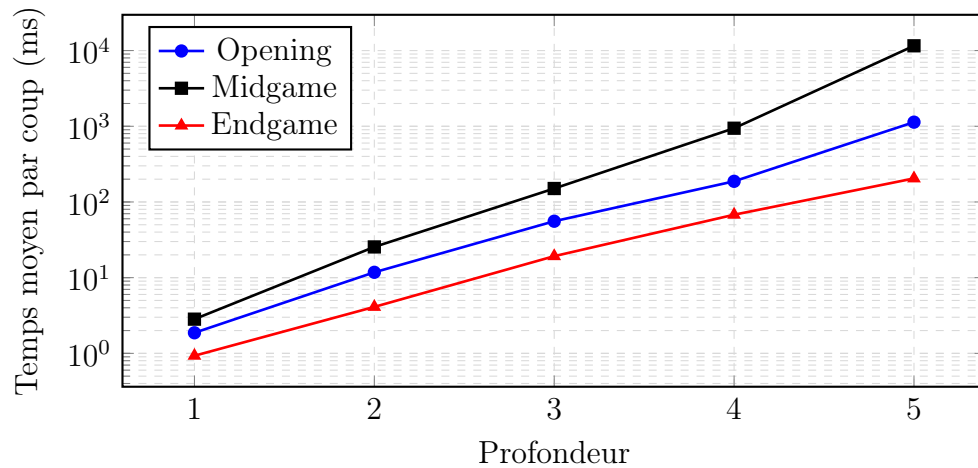


FIGURE 7 – Temps moyen de calcul par coup en fonction de la profondeur, par phase (échelle logarithmique).

La Figure 7 montre une croissance rapide du temps de calcul avec la profondeur. Le milieu de partie domine le coût (beaucoup plus de branches), tandis que l'ouverture et la fin de partie sont moins coûteuses. Augmenter la profondeur améliore la qualité des coups, mais augmente fortement le temps de calcul.

6.2 Taux de victoire IA vs Random

Des parties opposant l'IA à un joueur aléatoire ont été organisées afin d'évaluer la robustesse de la fonction d'évaluation.

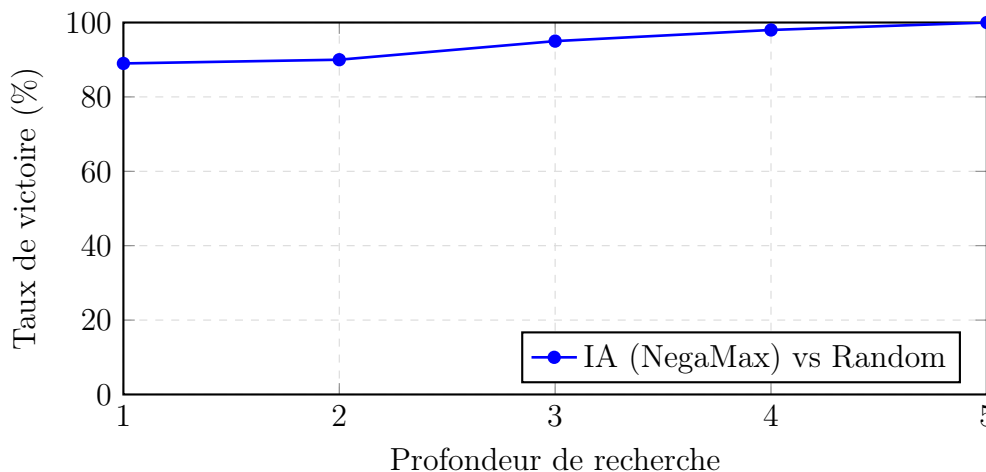


FIGURE 8 – Taux de victoire % IA vs Random (100 parties par profondeur).

La Figure 8 met en évidence une augmentation du taux de victoire avec la profondeur. Dès la profondeur 1 l'IA dépasse largement un joueur aléatoire, puis les gains réduisent et on observe un effet de palier.

6.3 Comparaison entre profondeurs successives

Des parties opposant l'IA à elle-même à une profondeur successive ont été organisées afin d'évaluer les gains de performances à chaque saut de profondeur.

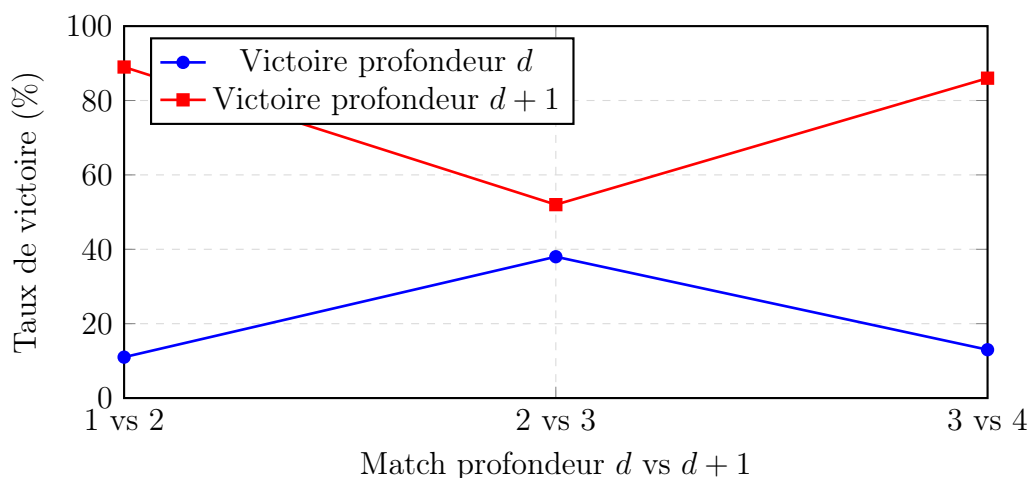


FIGURE 9 – Taux de victoire (100 parties) entre profondeurs successives (d vs $d+1$).

La Figure 9 montre que l'amélioration due à la profondeur n'est pas linéaire, en effet certaines augmentations de profondeur (1→2, 3→4) apportent un gain beaucoup plus important que d'autres (2→3), ce qui met en évidence des seuils de progression propres à Reversi.

6.4 Consommation mémoire

La consommation mémoire a été analysée à l'aide de tracemalloc.

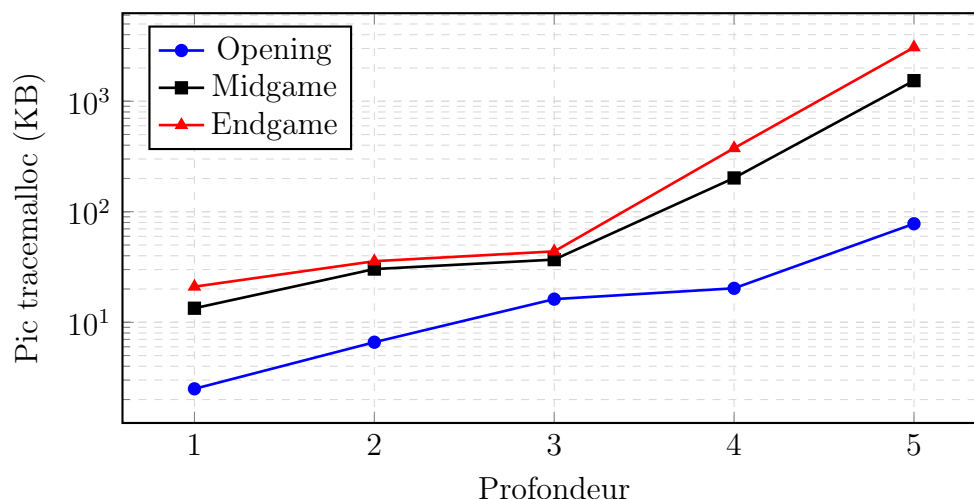


FIGURE 10 – Pic d'allocations Python mesuré par `tracemalloc` en fonction de la profondeur (échelle logarithmique).

La Figure 10 montre une augmentation des allocations internes avec la profondeur en particulier au milieu et en fin de partie. Une partie de ces allocations étant due au chargement

des bibliothèques, les variations restent faibles.

7 Tournoi IA vs IA

7.1 Méthodologie expérimentale

Un tournoi a été organisé entre plusieurs stratégies à profondeur fixe. Chaque confrontation a été répétée $n = 100$ fois, avec alternance du joueur qui commence pour obtenir des résultats équitables.

7.2 Résultats

Les résultats du tournoi sont présentés sous forme de tableau, chaque case indiquant le taux de victoire de l'IA en ligne contre celle en colonne. Cette représentation permet de comparer directement la robustesse des différentes stratégies.

	Default	Defense	Corners	Mobility	End	Total
Default	–	90	70	0	60	40
Defense	10	–	92	0	28	32.5
Corners	30	8	–	0	18	14.0
Mobility	100	100	100	–	100	100.0
End	40	72	82	0	–	48.5

TABLE 2 – Matrice de tournoi IA vs IA (profondeur $d = 3$, $n = 100$ parties par confrontation, alternance du joueur qui commence). Les matchs nuls ne sont pas comptabilisés dans les pourcentages.

On observe que la stratégie Mobility sort vainqueur du tournoi avec un taux de victoire de 100.0%, suivit de End (48.5%), Default (40.0%), Defense (32.5%) et Corners (14.0%).

8 Conclusion

Ce projet avait pour objectif de développer une IA compétitive pour Reversi, utilisable en modes Joueur vs Joueur, Joueur vs IA et IA vs IA. Le jeu a été entièrement modélisé (coups légaux, retournement des pions, fin de partie, etc.) et l'architecture modulaire facilite les évolutions futures.

La stratégie repose sur NegaMax avec élagage Alpha-Bêta et une fonction d'évaluation multi-critères. Les expérimentations confirment :

- une augmentation du taux de victoire avec la profondeur
- une croissance rapide du temps de calcul (surtout en milieu de partie)

— une consommation mémoire maîtrisée

Le move ordering est l'optimisation la plus impactante sur la performance, car il améliore fortement l'efficacité de l'élagage Alpha-Bêta dans les phases complexes.

En modifiant le poids des paramètres de la fonction heuristique, on obtient différentes stratégies qui ont été mises en compétition où la **Mobility** ressort vainqueur. Cela montre que le nombre de choix possible est le paramètre qui influe le plus sur la performance. Les valeurs étant déterminées de manière arbitraire, cela mériterait une amélioration via un réseau de neurones où les poids s'ajusteraient automatiquement pour optimiser son efficacité face à de nouvelles stratégies.

9 Bibliographie

Références

- [1] Wikipédia, *Algorithme minimax*, https://fr.wikipedia.org/wiki/Algorithme_minimax
- [2] Wikipédia, *Negamax*, <https://en.wikipedia.org/wiki/Negamax>
- [3] Wikipédia, *Classification des jeux*, https://fr.wikipedia.org/wiki/Classification_des_jeux
- [4] Wikipédia, *Jeu à somme nulle*, https://fr.wikipedia.org/wiki/Jeu_%C3%A0_somme_nulle
- [5] Fédération Française d'Othello, *Règles officielles du jeu Othello*, <https://www.ffothello.org/othello/regles-du-jeu/>
- [6] S. Russell, P. Norvig, *Artificial Intelligence : A Modern Approach*, Pearson Education, 3rd edition, 2010.
- [7] J. Pearl, *Heuristics : Intelligent Search Strategies for Computer Problem Solving*, Addison-Wesley, 1984.
- [8] M. Buro, *Improving Heuristic Mini-Max Search by Supervised Learning*, Artificial Intelligence Journal, 1999.
- [9] Textualize, *Rich : Python library for rich text and beautiful formatting*, <https://github.com/Textualize/rich>